Unidad 4: Introducción a la Programación en Assembler

TEÓRICO

Universidad Tecnológica Nacional

Facultad Regional Córdoba

Cátedra: Arquitectura de Computadoras (ACO)

CICLO LECTIVO 2026

Autores: María Alejandra Paz Menvielle

Félix Domingo Piozzi

INDICE

4.1. Conceptos introductorios 3
4.1.1. ¿Por qué estudiar Assembler? 3
4.1.2. Traductores 3
Ensambladores 4
Compiladores 5
Intérpretes 6
4.2. Registros Internos del Procesador 6
4.2.1. Registros de Estado o Registro de Banderas (flags) 7
4.2.2. Registros de propósito general 8
4.2.3. Registros de segmento 9
4.2.4. Registros Apuntadores o Punteros 10
4.2.5. Registros Índice 10
4.3. Modos de direccionamiento de la familia Intel 10
4.3.1 Modo de Direccionamiento Inmediato 11
4.3.2. Modo de direccionamiento Directo 11
4.3.3 Modo de direccionamiento Indirecto por Registro 12
4.3.4 Modo de Direccionamiento Registro 12
4.3.5 Modo de Direccionamiento Implícito o Inherente 13
4.3.6 Modo de direccionamiento Con Desplazamiento: Relativo 13
Tipos de Operaciones 14
4.3 Ejemplos en Ensamblador x86: *Estructura de un programa* 19
4.3.1 Ejemplo de elementos del Lenguaje Ensamblador 22
4.4. Enlazadores y Cargadores 24
AUTOESTUDIO 27
BIBLIOGRAFIA 30

Unidad 4: Introducción a la programación en Assembler.

4.1. Conceptos introductorios

4.1.1. ¿Por qué estudiar Assembler?

El assembler fue el primer lenguaje simbólico creado para permitir escribir programas desde el teclado de un computador (salida en código ASCII), ello suponía la existencia de un programa ensamblador para pasar los símbolos que estaban en código ASCII a código de máquina.

Según Stallings [1], el *ensamblador (assembler) es un programa traductor que convierte un programa escrito en lenguaje ensamblador (representación simbólica del lenguaje máquina) en código máquina ejecutable por el procesador*.

El Lenguaje de o Código de Máquina está ubicado en el nivel más bajo (Figura 4.1), donde el microprocesador responde a un listado de operaciones que se denomina programa máquina. Este listado de operaciones comprende un programa almacenado en memoria, que el micro debe ejecutar en el orden en que se encuentran.

Cada procesador posee su correspondiente lenguaje assembler y su ensamblador. Por lo general los procesadores modernos de un mismo fabricante conservan las instrucciones de assembler de modelos anteriores. Así, por ejemplo, los nuevos procesadores de la familia Intel tienen sus instrucciones del antiguo 80286, es por ello que se hace referencia a la familia x86, por razones de compatibilidad. Si bien cada fabricante desarrolla su propio lenguaje ensamblador, en este trabajo se desarrollará el assembler de la familia Intel como lenguaje representativo.

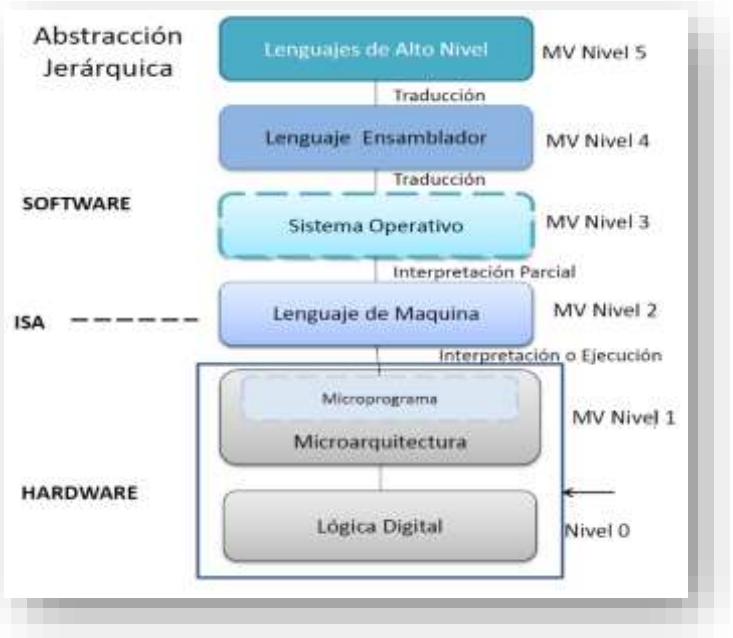


Figura 4.1 Abstracción jerárquica de los programas que intervienen en el computador

4.1.2. Traductores

Son programas cuya función es convertir los programas escritos por el usuario en lenguaje simbólico a lenguaje de máquina. Un programa escrito en lenguaje simbólico se denomina

fuente, mientras que un programa en lenguaje de maquina se denomina ejecutable. Las instrucciones del programa fuente constituyen la entrada al traductor de lenguaje, las salidas serán las instrucciones del programa ejecutable. La traducción comprende el análisis del léxico y la sintaxis de cada instrucción o sentencia, así como las referencias lógicas a las que apunta cada instrucción.

Se necesita un traductor para cada lenguaje simbólico que se utilice. Se destacan tres tipos de traductores de lenguaje: ensambladores, compiladores e intérpretes.

Ensambladores

Para transformar la descripción del lenguaje humano del programa a código máquina, se estableció un sencillo lenguaje de programación que utiliza palabras y frases para representar los códigos máquina del microprocesador. Se encarga de traducir un archivo fuente escrito en un lenguaje Assembler a un archivo cuyas instrucciones estén en código máquina, que es ejecutable directamente por el procesador para el que se creó (Figura 4.2).

Generalmente una frase en lenguaje ensamblador constará de 1 a 3 bytes de código máquina y está constituida por un único nemotécnico de 3 letras.
La programación en lenguaje ensamblador es un método de «humanizar» las operaciones de un microprocesador, por medio de un lenguaje simbólico, que permite la escritura de programas a nivel *código de máquina,* es decir *un código binario decodificable por el procesador, pero imposible de escribir en la práctica*.
Todo procesador, grande o pequeño posee su lenguaje máquina y, por lo tanto, su lenguaje simbólico de máquina.

Todo lenguaje tiene un conjunto de reglas que el programador debe respetar. Un programa Assembler está organizado con una secuencia lógica de líneas de programa, se acostumbra a dividir las sentencias del lenguaje ensamblador en 4 campos: Etiqueta, nemotécnico, operando y comentarios, a continuación, se explica cada una:

- Etiqueta: No se utiliza siempre. Corresponde a un tipo especial de modo de direccionamiento.
- Nemotécnico: es un conjunto de letras, números y símbolos que representan una instrucción. Este campo indica al programa ensamblador esencialmente la operación que se va a realizar.
- Operando: Contiene información sobre registros, datos o direcciones asociados con la operación. Corresponden a la forma en que se van a mover los datos, desde operando destino hacia el operando fuente.
- Comentario: no lo evalúa el ensamblador, solo lo imprime. Es de ayuda para comprender que ocurre en el programa

Con la información de los campos nemotécnico y operando, el programa ensamblador puede generar el código correcto en lenguaje máquina.

Cuando el programa es complejo suele dividirse en módulos que se programan en forma independiente y al momento de traducción pueden estar alojados en zonas diferentes de memoria, que se ensamblan y se ponen a punto de manera independiente del proceso de traducción. Es necesario juntar los módulos de manera de organizar un solo programa ejecutable, donde las instrucciones se ejecuten una detrás de otra.

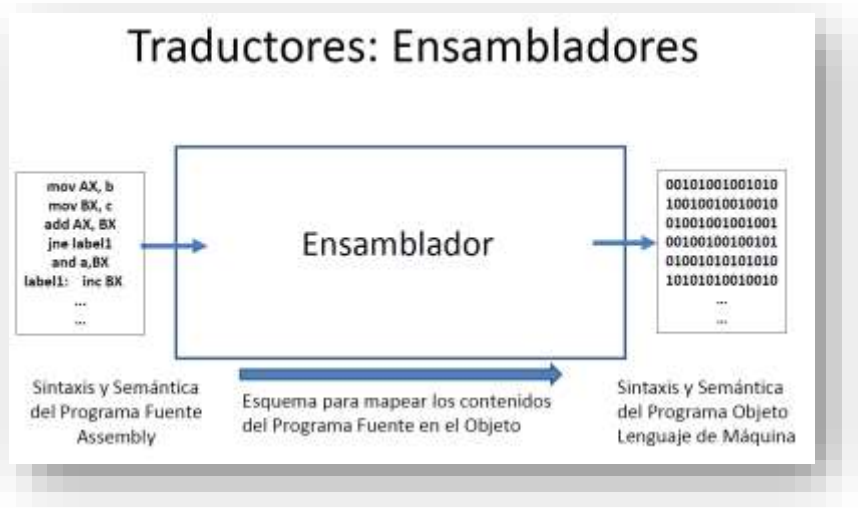


Figura 4.2 Funcionamiento de un Ensamblador

Compiladores

Un compilador es un *traductor especial que toma un programa escrito en lenguaje de alto nivel (programa Fuente) y lo traduce a una representación escrito en lenguaje de bajo nivel (programa Objeto)* [11].

En la Figura 4.3 puede verse el proceso de compilación, donde:

- LAN: Lenguaje de Alto Nivel utilizado en la programación (Figura 4.5).
- El Front End es la parte visual e interactiva de una aplicación o sitio web con la que el usuario interactúa directamente (diseño, menús, botones).
- El Back End es la parte oculta, compuesta por servidores, bases de datos y la lógica funcional que hace que la aplicación funciones.
- RI es la Representación Intermedia que es una estructura de datos o código utilizado internamente por un compilador para representar el código fuente de manera independiente de la máquina, permitiendo optimizaciones generales antes de convertirlo al lenguaje máquina final específico.

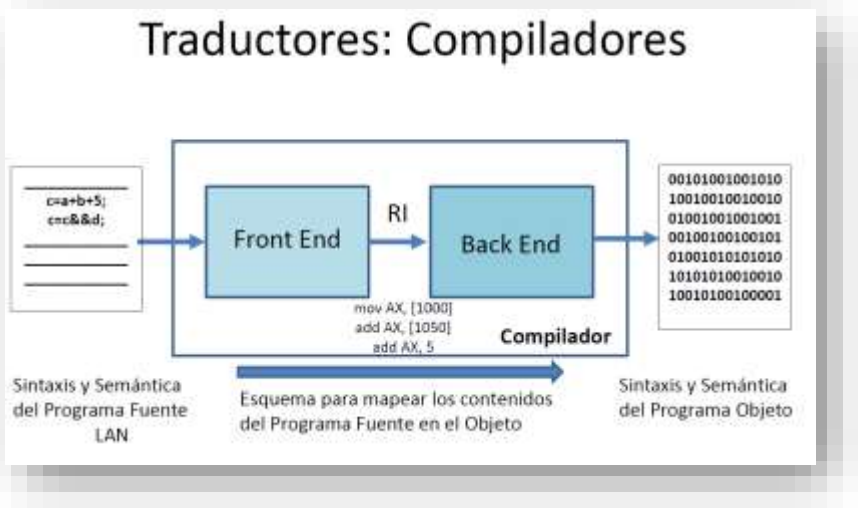


Figura 4.3 Proceso de Ccompilación

Intérpretes

Giro et all [11], define al intérprete como *un programa que interpreta y ejecuta las sentencias de un programa fuente una tras otra, sin generar un programa objeto*.

En la figura 4.4 puede verse la comparación del tratamiento de un programa a través de la compilación y la interpretación.



Figura 4.4 Tratamiento de un programa con la Compilación y la Interpretación [11]



Figura 4.5 Lenguajes de Alto Nivel utilizados en la Programación

4.2. Registros Internos del Procesador

Los microprocesadores están compuestos de Registros internos para realizar el procesamiento. Los registros son direccionables por medio de un nombre (AX, BX, SI, DI, etc.), además en ellos, por convención, los bits se numeran de derecha a izquierda.

Sus funciones principales son:

- Guardar la información de manera temporal durante las operaciones.
- Controlar instrucciones en ejecución
- Manejar direccionamiento de memoria
- Proporcionar capacidad aritmética.

William Stallings [1] presenta, en las arquitecturas de los procesadores de la familia x86, los registros podrán como se muestran en la Figura 4.6.

64 bits	32 bits	16 bits	8 bits	8 bits
RAX	EAX	AX	AH	AL
RBX	EBX	BX	BH	BL
RCX	ECX	CX	CH	CL
RDX	EDX	DX	DH	DL
RSI	ESI	SI	No disponible	
RDI	EDI	DI	No disponible	
RSP	ESP	SP	No disponible	
RBP	EBP	BP	No disponible	

Figura 4.6 Registros de los procesadores de la familia x86

Los registros de 32 bits se llaman Extended EAX. Si a los registros de 64 bits se agrega la R al nombre REAX o RAX:

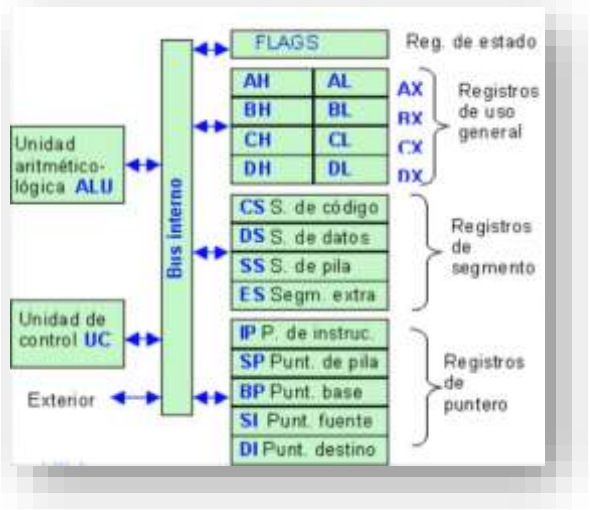


Figura 4.7 Registros internos de los procesadores de la familia Intel x86 de 16 bits

A continuación, se describen cada uno de los registros internos de los procesadores de la familia Intel x86, que pueden verse en la Figura 4.7 [1]. En este trabajo se describen los registros de 16 bits.

4.2.1. Registros de Estado o Registro de Banderas (flags)

Los Registros de estado sirven para indicar el estado actual de la máquina y el resultado del procesamiento. Muchas instrucciones que piden comparaciones y aritmética cambian el estado de las banderas, algunas cuyas instrucciones pueden realizar pruebas para determinar la acción subsecuente.

- **Overflow (desbordamiento).** Indica desbordamiento de un bit de orden alto (más a la izquierda) después de una operación aritmética.

- **D (dirección).** Designa la dirección hacia la izquierda o hacia la derecha para mover o comparar cadenas de caracteres.

- **I (interrupción).** Indica que una interrupción externa, como la entrada desde el teclado, sea procesada o ignorada.

- **T (trampa).** Permite la operación del procesador en modo de un paso. Los programas depuradores, como el DEBUG, activan esta bandera de manera que usted pueda avanzar en la ejecución de una sola instrucción a un tiempo, para examinar el efecto de esa instrucción sobre los registros de memoria.

- **S (signo).** Contiene el signo resultante de una operación aritmética (0 = positivo y 1 = negativo).

- **Z (cero).** Indica el resultado de una operación aritmética o de comparación (0 = resultado diferente de cero y 1 = resultado igual a cero).

- **A (acarreo auxiliar).** Contiene un acarreo externo del bit 3 (primer nibble o primeros 4 bits contando desde la izquierda) en un dato de 8 bits para aritmética especializada.

- **P (paridad).** Indica paridad par o impar de una operación en datos de 8 bits de bajo orden (es decir más a la derecha).

- **C (acarreo).** Contiene el acarreo de orden más alto (o sea más a la izquierda) después de una operación aritmética; también lleva el contenido del último bit en una operación de corrimiento o de rotación.

Las banderas están en el Registro de Estado en las siguientes posiciones (Figura 4.8):



Figura 4.8 Posición de los bits en el Registro de Estado.

4.2.2. Registros de propósito general

Las arquitecturas x86 inicialmente poseían 4 registros de propósito general de 16 bits: AX, BX, CX y DX.

Son los únicos que se pueden direccionar como una palabra o como una parte de un byte. El ultimo byte de la izquierda es la parte "alta", y el ultimo byte de la derecha es la parte "baja".

- **Registro AX:** el registro AX, el acumulador principal, conserva el resultado después de operaciones que implican entrada/salida y la mayor parte de la aritmética. Por ejemplo, las instrucciones para multiplicar, dividir y traducir suponen el uso del AX.

- **Registro BX:** el BX es conocido como el registro base ya que es el único registro de propósito general que puede ser índice para direccionamiento indexado. Guarda direcciones de registros de memoria. También es común emplear el BX para cálculos.

- **Registro CX:** CX es conocido como el registro contador. Se utiliza para realizar operaciones de sumas y restas para contar.

- **Registro DX:** el DX es conocido como el registro de datos. Algunas operaciones de entrada/salida requieren su uso. Guarda la parte más significativa del producto luego de una multiplicación, y el dividendo antes de una división.

4.2.3. Registros de segmento

La memoria principal está organizada en distintos segmentos, cada uno con un propósito, es decir que un segmento es una parte de la memoria en la que se almacenan datos, programas, contenido de la pila y otros. Cada segmento inicia en una posición de memoria (Véase Figura 4.9). En la familia x86 de Intel los segmentos son de 64 kb, y cada uno posee una dirección de memoria inicial que es almacenada en su correspondiente registro.
Por su parte, la pila o Stack, es un área especial de la RAM donde la CPU guarda información temporal mientras ejecuta programas. Los punteros SP (Stack pointer) y BP (Base Pointer) siempre trabajan sobre esa pila, por eso utilizan automáticamente el segmento de pila o SS (Stack Segment).
De manera similar, los registros SI (Source Index), DI (Destination Index) y BX, se utilizan principalmente para acceder a datos almacenados en memoria, por eso normalmente utilizan el segmento de datos o DS (Data Segment) [12].

Los registros de los 4 segmentos principales son:

- **Registro Segmento de Código o Code Segment-CS:** El DOS almacena la dirección inicial del segmento de código de un programa.

- **Registro Segmento de Datos o Data Segment-DS:** Almacena la dirección inicial de un segmento de datos estáticos del programa.

- **Registro Segmento de Pila o Stack Segment-SS:** permite la colocación en memoria de una pila, para almacenamiento temporal de direcciones y datos. El DOS almacena la dirección de inicio del segmento de pila de un programa en el registro Segmento de pila.

- **Registros Segmento Extra o Extra Segment-ES:** Algunas operaciones del microprocesador para manejo de cadenas de caracteres (datos de caracteres) utilizan el registro extra de segmento para manejar el direccionamiento de memoria. En este contexto, el registro Segmento Extra está asociado con el registro DI (índice).

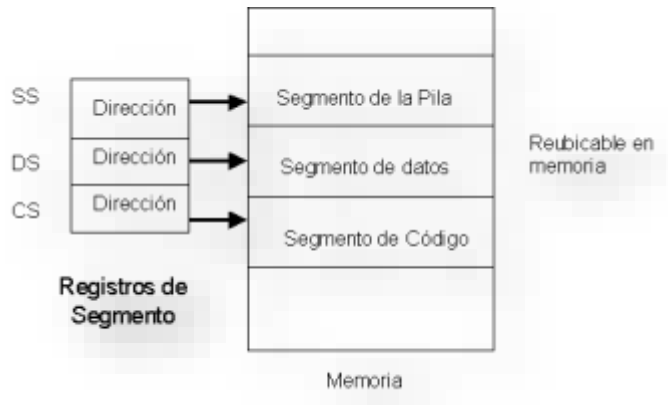


Figura 4.9 Registros de segmento

4.2.4. Registros Apuntadores o Punteros

Guardan las direcciones de los punteros e índices. Son los siguientes:

- **IP (Instruction pointer):** El registro puntero a la próxima instrucción a ejecutarse (IP) de 16 bits contiene el desplazamiento de dirección de la siguiente instrucción que se ejecuta (Contador de programa -PC). El IP está asociado con el registro CS ya que el IP indica la instrucción actual dentro del segmento de código que se está ejecutando actualmente.

- **SP (stack pointer).** El apuntador de la pila de 16 bits está asociado con el registro SS (Stack Segment). El SP apunta a un sector de la memoria en el que, ya sea por llamada a una subrutina o una interrupción el microprocesador guarda la información necesaria (registro IP y otros datos) para saber dónde volver para continuar el procesamiento cuando la subrutina o interrupción haya finalizado. Guarda la información en forma FIFO O LIFO dependiendo del microprocesador.

- **BP (base pointer).** El registro apuntador de base, de 16 bits facilita la referencia de parámetros, los cuales son datos y direcciones transmitidos vía pila. Está asociados con el registro SS (stack segment) y permite acceder a los datos en el segmento de la pila.

4.2.5. Registros Índice

Los registros Índice Fuente (Source Index - SI) e Índice Destino (Destination Index - DI) están disponibles para direccionamiento indexado y para sumas y restas. Corresponden a la forma de acceso mediante registro indirecto a la memoria.

- **Registro SI.** El registro índice fuente es de 16 bits es requerido por algunas operaciones con cadenas (de caracteres). En este contexto, el SI está asociado con el registro DS.

- **Registro DI.** El registro índice destino también es requerido por algunas operaciones con cadenas de caracteres. En este contexto, el DI está asociado con el registro ES.

4.3. Modos de direccionamiento de la familia Intel

Una de las partes del Registro de Instrucción de la Unidad de Control está asociada al operando. La técnica que se utiliza para encontrar el valor del operando se llama Direccionamiento.

Cada procesador presenta diferentes formas o modos para realizar el direccionamiento. Entonces se define *Modo de direccionamiento como la técnica mediante la cual la instrucción accede al/los operando/s*. Es posible que se requiera obtener el Operando destino y el Operando origen.

Los diversos modos de direccionamiento están asociados a cada microprocesador, fueron tratados en la Unidad 3 – Arquitectura Convencional y Avanzada. A continuación, se revisarán y se darán ejemplos de los siguientes:

- Inmediato
- Directo
- Indirecto por Registro o Indirecto a Memoria
- Por Registro

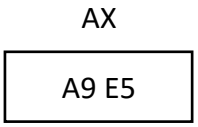
- Implícito o Inherente
- Con desplazamiento: Relativo

4.3.1 Modo de Direccionamiento Inmediato

El operando fuente aparece en la instrucción, permite pasar una constante a un registro.

Ejemplo:

06B3:0100 MOV AX, A9E5



Ocupa 3 bytes

06B3:0100 B8 --- B8 es el código de la operación (MOV)

06B3:0101 E5

06B3:0102 A9

4.3.2. Modo de direccionamiento Directo

El operando fuente contiene la dirección de memoria donde se encuentra el dato. Su valor se escribe entre corchetes "[ ]", (Figura 4.10).

Ejemplo:

06B3:0100 MOV BX, [A5F2]

Ocupa 4 bytes

06B3:0100 8B

06B3:0101 1E

06B3:0102 A5

06B3:0103 F2

El código de operación es 8B1E está en las celdas de memoria 06B3:0100 y 06B3:0101, en las celdas siguientes 06B3:0102 y 06B3:0103 está la dirección del operando A5F2.

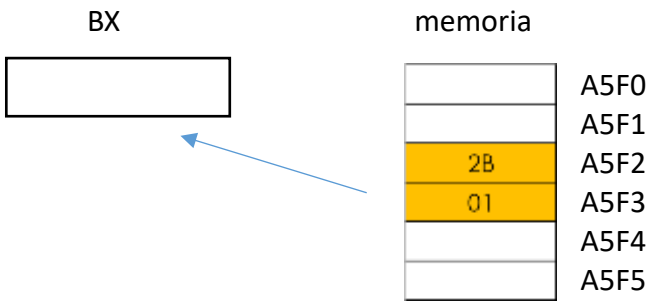


Figura 4.10 Ejemplo de modo de direccionamiento Directo

Otro ejemplo de este MD es de la forma: MOV [2000], BX.

4.3.3 Modo de direccionamiento Indirecto por Registro

El campo de dirección de la instrucción se da indirectamente a través de un registro, como SI, DI o BX. Las instrucciones en este MD permiten solucionar el problema de recorrer uno tras otro los datos contenidos en posiciones sucesivas de memoria, como es el caso de una lista o vector (Figura 4.11). Ejemplo:

06B3:0100 MOV AX, [BX]

Ocupa 2 bytes

06B3:0100 8B

06B3:0101 07

En esta instrucción el código de operación es 8B07 y está en las celdas de memoria cuya dirección es 06B3:0100 y 06B3:0101. El dato u operando que es el valor a ser asignado al registro AX, está en la dirección contenida en el registro BX, nótese que también para indicar que el registro BX contiene la dirección del dato y no el dato el registro BX se pone entre corchetes.

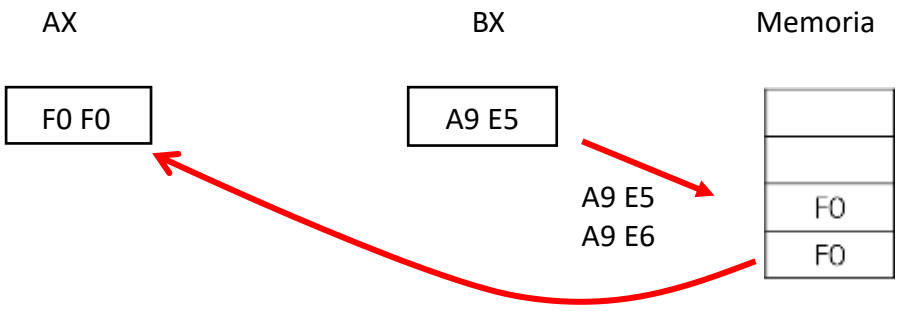


Figura 4.11 Ejemplo modo de direccionamiento Indirecto

4.3.4 Modo de Direccionamiento Registro

Indica que el dato u operando está contenido en uno de los registros internos de propósito general del CPU, y el resultado va también a un registro de la CPU (Figura 4.12).

Ejemplo:

06B3:0100 MOV AX, BX

Ocupa 2 bytes.

06B3:0100 89

06B3:0101 D8

El código de operación es 89D8 y está en las celdas de memoria cuya dirección es 06B3:0100 y 06B3:0101. El dato u operando, que es el valor que debe ser asignado al registro AX, está contenido en el registro BX.

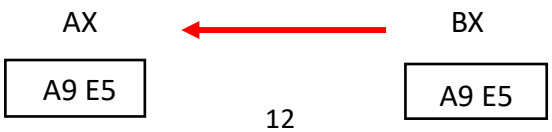


Figura 4.12 Ejemplo modo de direccionamiento Registro

Otro ejemplo de este MD es la instrucción DEC CX que decrementa en uno el valor contenido en el registro CX.

4.3.5 Modo de Direccionamiento Implícito o Inherente

Se especifican los operandos en forma implícita en la definición de la instrucción, o código de operación o puede no tener dato porque la instrucción no lo necesita, es decir que el dato está en la instrucción.

Ejemplo:

06B3:0100 INT 3

Ocupa 1 byte

06B3:0100 CC

Esta instrucción finaliza una secuencia de instrucciones de un programa. El código de operación es CC, al ser una acción para terminar la ejecución de un programa o grupo de acciones, no hay dato u operando, por lo que el mismo código de operación tiene todo lo necesario para ejecutarla.

4.3.6 Modo de direccionamiento Con Desplazamiento: Relativo

La instrucción JNZ como cualquier otra de salto condicional “short” (corto) utiliza en modo de direccionamiento Relativo. En un salto corto a lo sumo se puede saltar a una instrucción que esté -128 posiciones hacia atrás, o +127 posiciones hacia adelante (es decir 80_H), que son los números que se pueden representar en un byte. Si no se respetan los límites citados, el ensamblador rechazará la instrucción de salto.

El término “relativo” implica que la dirección de la instrucción a la que se salta (si se cumple la condición) se calcula sumando un número a la dirección de la instrucción que sigue a la instrucción de salto. Es decir que la dirección hacia donde hay que saltar está en relación con esta última dirección, la cual está apuntada por el IP (Instruction Pointer o Contador de Programa), por ser ésta la dirección de la próxima instrucción a ejecutar luego de la de salto. En un salto hacia atrás dicho número es negativo, siendo positivo si el salto es hacia adelante. Ejemplo:

06B3:0110 JNZ 0109 ó 06B3:0110 75F7

en la dirección 06B3:0110 está la instrucción JNZ 0109 cuyo objetivo es saltar hacia atrás a la dirección 06B3:0109.

Para ampliar lo visto, recuerde que los operandos de origen y resultado pueden estar en una de cuatro áreas:

- Memoria principal o virtual: Al igual que con las siguientes referencias de instrucciones, la memoria principal o virtual se debe proporcionar la dirección de memoria.
- Registro del procesador: con raras excepciones, un procesador contiene uno o más registros que pueden ser referenciados por las instrucciones de la máquina. Si sólo

existe un registro, la referencia a él puede estar implícita. Si existe más de un registro, entonces a cada registro se le asigna un nombre o número único, y la instrucción debe contener el número del registro deseado.

- Inmediato: El valor del operando está contenido en un campo de la instrucción que está siendo ejecutado.
- Dispositivo de E/S: La instrucción debe especificar el módulo de E/S y el dispositivo para la operación. Si se utiliza E/S asignada en memoria, esta es simplemente otra dirección de memoria principal o virtual.

Tipos de Operaciones

Como se trató en la Unidad 3, el número de códigos de operación diferentes varía mucho de una máquina a otra. Sin embargo, los mismos tipos generales de operaciones se encuentran en todas las máquinas. Una útil y típica categorización es la que se explica a continuación:

- Transferencia de datos
- Aritméticas
- Lógicas
- E/S
- Control o de control de flujo
- Otros tipos de instrucciones

Transferencia de datos

En términos de acción del procesador, las operaciones de transferencia de datos son quizás las más simples. Si tanto el origen como el destino son registros, entonces el procesador transferirá los datos de un registro a otro; esta es una operación interna del procesador. Si uno o ambos operandos están en la memoria, entonces el procesador debe obtener ese dato y transferirlo según el modo de direccionamiento utilizado.

Aritméticas

La mayoría de las máquinas proporcionan las operaciones aritméticas básicas de sumar, restar, multiplicar, y dividir. Estos se proporcionan invariablemente para números enteros con signo (de punto fijo).

A menudo también se proporcionan para números decimales empaquetados y de punto flotante.

Otras operaciones posibles incluyen una variedad de instrucciones de un solo operando; por ejemplo:

- Absoluto: toma el valor absoluto del operando.
- Negar: niega el operando.
- Incremento: Suma 1 al operando.
- Decremento: Resta 1 del operando.

La ejecución de una instrucción aritmética puede implicar operaciones de transferencia de datos y posicionar operandos para la entrada a la ALU y entregar la salida de la ALU.

Lógicas

La mayoría de las máquinas también ofrecen una variedad de operaciones para manipular bits individuales de una palabra u otras unidades direccionables, lo que a menudo se denomina "juego de bits", ellos son basados en operaciones booleanas.

La operación NOT se invierte un bit. AND, OR y OR exclusivo (XOR) son las funciones lógicas más comunes con dos operandos. EQUAL es una prueba binaria útil.

Estas operaciones lógicas se pueden aplicar bit a bit a unidades de datos lógicos de n bits. Por ejemplo, si dos registros contienen los datos (R1) = 10100101 y (R2) = 00001111

Entonces: (R1) AND (R2) = 00000101

donde la operación AND se puede utilizar como una máscara que selecciona ciertos bits en una palabra y pone a cero el resto de los bits.

Además de las operaciones lógicas bit a bit, la mayoría de las máquinas proporcionan una variedad de funciones de *cambio y funciones de rotación*. Con un desplazamiento lógico, los bits de una palabra se desplazan hacia la izquierda o hacia la derecha. En un extremo, el bit que se desplazó hacia afuera, se pierde. En el otro extremo, se desplaza un 0. Los desplazamientos lógicos son útiles principalmente para aislar aislar campos dentro de una palabra. Los 0 que se desplazan hacia una palabra desplazan información no deseada que se desplaza del otro extremo hacia afuera y se pierden.

La operación de *desplazamiento aritmético* trata los datos como un entero con signo y no desplaza el bit de signo. En un desplazamiento aritmético a la derecha, el bit de signo se replica en la posición del bit a su derecha. En un desplazamiento aritmético hacia la izquierda, se realiza un desplazamiento lógico hacia la izquierda en todos los bits, pero el bit de signo, se conserva. Estas operaciones pueden acelerar ciertas operaciones aritméticas. Con números en notación en complemento a dos, un desplazamiento aritmético a la derecha corresponde a una división por 2, con truncamiento para números impares. Tanto una aritmética izquierda desplazamiento y un desplazamiento lógico hacia la izquierda corresponden a una multiplicación por 2 cuando no hay desbordamiento. Si se produce un desbordamiento, las operaciones aritméticas y lógicas de desplazamiento a la izquierda producen diferentes resultados, pero el desplazamiento aritmético a la izquierda conserva el signo del número. Debido a la posibilidad de desbordamiento, muchos procesadores no incluyen esta instrucción, incluidos PowerPC e Itanium. Otros, como el IBM EAS/390, ofrecen la instrucción.

Curiosamente, la arquitectura x86 incluye un desplazamiento aritmético hacia la izquierda, pero lo define como idéntico a un desplazamiento lógico a la izquierda.

Las operaciones de *rotación o desplazamiento cíclico* preservan todos los bits en los que se opera. Un uso de una rotación es llevar cada bit sucesivamente al bit más a la izquierda, donde puede identificarse probando el signo de los datos (tratados como un número).

Al igual que con las operaciones aritméticas, las operaciones lógicas implican actividad ALU y pueden implicar operaciones de transferencia de datos. y rotar operaciones discutidas en esta subsección.

Entrada/Salida

En las instrucciones de entrada/salida hay una variedad de enfoques adoptados, incluyendo E/S programadas aisladas, mapeadas en memoria, E/S programadas, DMA y el uso de un procesador de E/S. Muchas implementaciones proporcionan sólo unas pocas instrucciones de E/S, con las acciones específicas especificadas por parámetros, códigos o palabras de comando.

Control o de control de flujo

Para todos los tipos de operación discutidos hasta ahora, la siguiente instrucción a realizar es el que sigue inmediatamente, en la memoria, a la instrucción actual. Sin embargo, una fracción significativa de las instrucciones de cualquier programa tienen como función cambiar la secuencia de ejecución de la instrucción. Para estas instrucciones, la operación realizada por el procesador es actualizar el contador del programa para que contenga la dirección de alguna instrucción en la memoria.

Hay varias razones por las que se requieren operaciones de transferencia de control.

Entre los más importantes se encuentran los siguientes:

1. En el uso práctico de las computadoras, es fundamental poder ejecutar cada instrucción más de una vez y quizás muchos miles de veces. Puede requerir miles o quizás millones de instrucciones para implementar una aplicación. Esto sería impensable si cada instrucción tuviera que escribirse por separado. Si se va a procesar una tabla o una lista de elementos, se necesita un bucle de programa. una secuencia de instrucciones se ejecuta repetidamente para procesar todos los datos.
2. Prácticamente todos los programas implican alguna toma de decisiones. Nos gustaría que la computadora pueda hacer una cosa si se cumple una condición, y otra cosa si se cumple otra condición. Por ejemplo, una secuencia de instrucciones calcula la raíz cuadrada de un número. Al comienzo de la secuencia, se prueba el signo del número. Si el número es negativo, no se realiza el cálculo, sino una condición de error y se informa.

Pasemos ahora a analizar las operaciones de transferencia de control más comunes. Se encuentra en conjuntos de instrucciones*: bifurcación, omisión y llamada a procedimiento*.

Instrucciones de Salto o bifurcación

Una instrucción de bifurcación o de salto, tiene como uno de sus operandos la dirección de la siguiente instrucción a ejecutar. Muy a menudo, la instrucción es una instrucción de salto condicional. Es decir, se realiza el salto (actualizar el contador de programa a la dirección igual especificada en el operando) sólo si se cumple una determinada condición. De lo contrario, se ejecuta la siguiente instrucción en secuencia (incremento del contador de programas como de costumbre). Una instrucción de salto en la que siempre se toma el salto es un salto incondicional.

Skip Instruction (Omitir Instrucciones)

Otra forma de instrucción de transferencia de control es la instrucción de omitir. La instrucción de omisión incluye una dirección implícita. Normalmente, la omisión implica que se salte una instrucción; por lo tanto, la dirección implícita es igual a la dirección de la siguiente instrucción más la longitud de una instrucción.

Debido a que la instrucción de omisión no requiere un campo de dirección de destino, es libre para hacer otras cosas. Un ejemplo típico es el incremento y omisión si el resultado es cero (ISZ).

Instrucciones de Llamada al Procedimiento

En cualquier punto del programa se puede invocar o llamar al procedimiento. El procesador recibe instrucciones para ir y ejecutar todo el procedimiento y luego regresar al punto desde que se realizó la llamada.

El mecanismo del procedimiento implica dos instrucciones básicas: una instrucción de llamada que se bifurca desde la ubicación actual al procedimiento, y una instrucción de devoluciónque

regresa del procedimiento al lugar desde donde fue llamado. Ambos de estos son formas de instrucciones de salto.

Debido a que nos gustaría poder llamar a un procedimiento desde una variedad de puntos, el procesador debe guardar de alguna manera la dirección de retorno para que el retorno pueda realizarse apropiadamente.

Hay tres lugares comunes para almacenar la dirección del remitente:

- En un Registro
- Al Inicio de la llamada al procedimiento
- En la parte superior de la pila

Un enfoque más general y potente es utilizar una pila. Cuando el procesador ejecuta una llamada, coloca la dirección de retorno en la pila. Cuando ejecuta un retorno, utiliza la dirección de la pila.

Otros tipos de instrucciones

Las instrucciones de control del sistema son aquellas que se pueden ejecutar sólo mientras el procesador se encuentra en un determinado estado privilegiado o está ejecutando un programa en un área privilegiada especial de la memoria. Normalmente, estas instrucciones están *reservadas para el uso del sistema operativo*.

Algunos ejemplos de operaciones de control del sistema, un sistema de control de la instrucción que puede leer o alterar un registro de control, otro ejemplo es una instrucción para leer o modificar una clave de protección de almacenamiento. Otro ejemplo es el acceso a Bloques de control de procesos en un sistema con multiprogramación.

Ejemplo del uso de ALU y registro de banderas

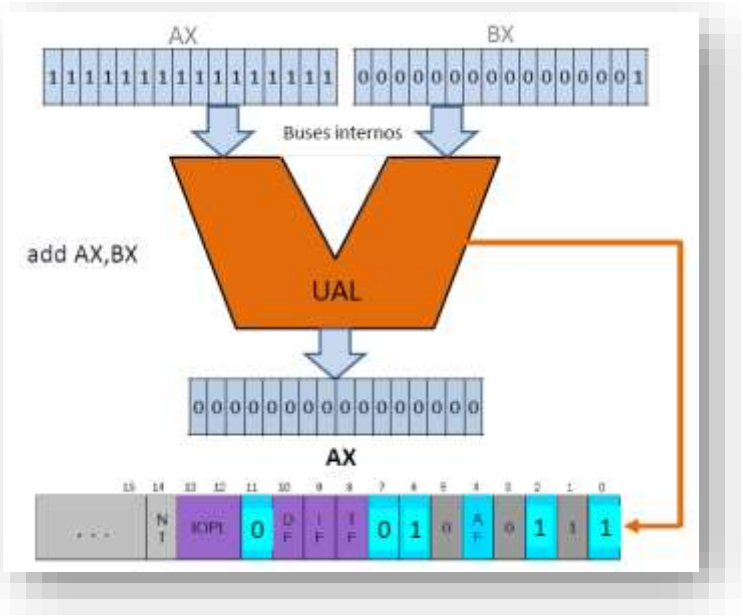


Figura 4.21 Un ejemplo de suma entre dos registros en la UAL y sus efectos en el Registro de Estado

En la figura 4.21 se ejemplifica la comparación de los registros AX y BX. La comparación internamente se realiza a través de la resta aritmética de los dos valores resultando tres valores posibles:

a) AX mayor que BX, el resultado será positivo, activándose las banderas de ZF=0 y SF=0
b) AX menor que BX, el resultado será negativo, activándose las banderas de ZF=0 y SF=1
c) AX = BX, el resultado será cero con lo que se activa la bandera ZF=0

En la figura 4.22 para restar BX de AX se obtiene su complemento a la base y se suma al valor del registro AX, obteniéndose en AX el resultado de esa operación y activándose las banderas correspondientes de ZF=0 y SF=1 en este caso por ser mayor el registro BX.

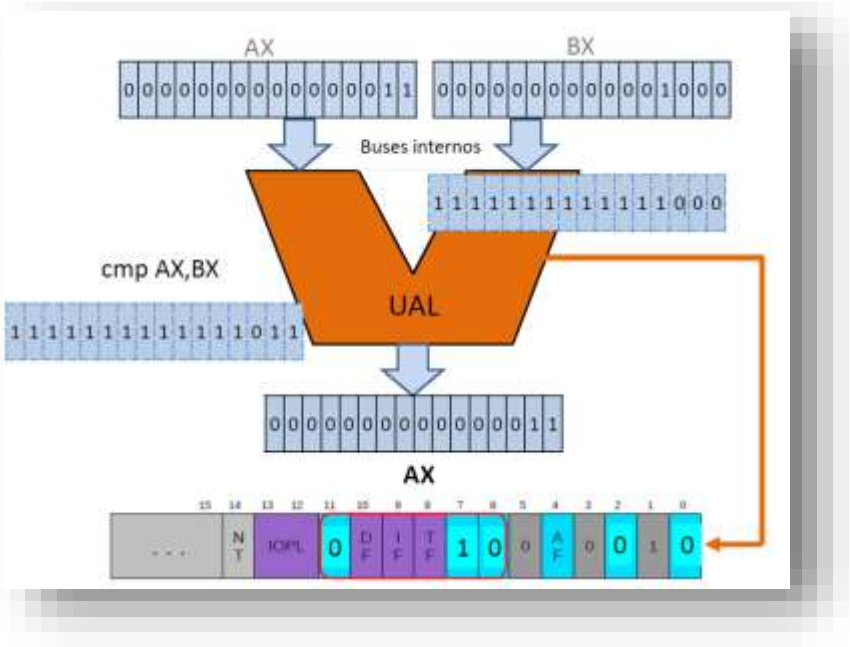


Figura 4.22 Ejemplo de comparación de dos registros AX y BX.

Instrucciones de Llamada a Procedimiento

La instrucción de Llamada a Procedimiento (Figura 4.27) previo a romper el flujo secuencial del programa y continuarlo en la subrutina o procedimiento llamada, guarda la dirección de la próxima instrucción secuencial (dirección de retorno) y luego ejecuta las instrucciones de la subrutina. Esta termina con una instrucción especial (ret) que vuelve a cargar el IP con la dirección de retorno y continua de esta forma con la ejecución secuencial del programa principal. La Llamada a Procedimiento tiene la siguiente estructura: CALL dir_del PROC

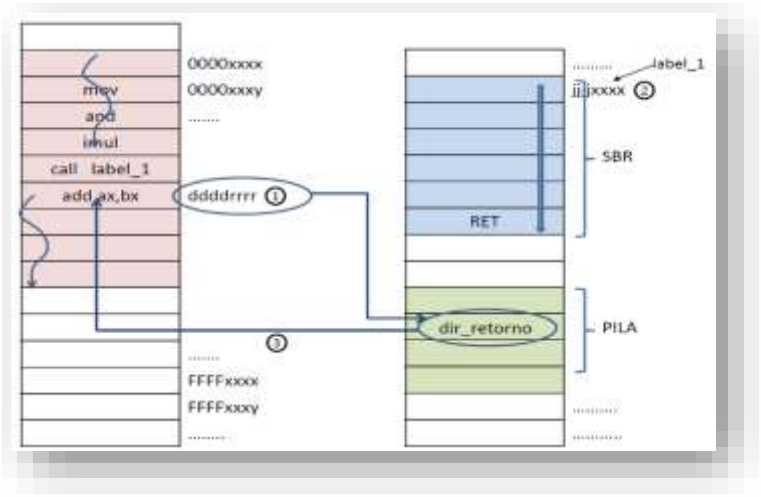


Figura 4.27 Proceso de llamada a procedimiento

4.3 Ejemplos en Ensamblador x86: *Estructura de un programa*

En la estructura de un Programa en Assembler, se coloca un Título (Title) de manera opcional, pero recomendable. Luego se realizan una serie de decalaraciones de procedimientos externos si los hubiera, a continuación, se describen las tres secciones directivas siguientes .stack, .data y .code que especifican segmentos definidos para configurar los mismos para realizar la programación, para luego realizar la carga del segmento de código propiamente dicho del programa (MAIN PROC) para finalmente escribir otros procedimientos o funciones. Recordar que el texto que sigue al carácter ";" son comentarios para la comprensión de cada línea.

La estructura de un programa en Assembler se organiza típicamente en tres secciones principales: datos (variables y constantes), bss (datos no inicializados) y código (instrucciones ejecutables). Se basa en un modelo secuencial que utiliza directivas, etiquetas, nemónicos y operandos para definir el inicio del programa, la memoria y las acciones del procesador.

Secciones Principales de un Programa Assembler

- *Sección de Datos (.data):* Se declaran variables y constantes inicializadas.
- *Sección BSS (.bss):* Reservada para variables no inicializadas o que se asignarán en tiempo de ejecución.
- *Sección de Código/Texto (.text):* Contiene las instrucciones que ejecutará el procesador y define el punto de entrada

Sintaxis de una Línea de Código

Cada línea se compone de hasta cuatro campos:

- *Etiqueta:* Marcador opcional para identificar posiciones de memoria (puntos de salto).
- *Operación (Nemónico):* La instrucción a ejecutar (ej. MOV, ADD, SUB).
- *Operandos:* Datos o registros sobre los que actúa la operación.
- *Comentarios:* Notas explicativas precedidas por el símbolo ;

La estructura de un Programa en Assembler en líneas generales contendría:

TITLE Programa ;esta línea es opcional

```
INCLUDE Nombre de Procedimiento externo
;las tres secciones directivas siguientes .stack, .data y .code especifican segmentos
.stack tamaño_de_la_pila
.data
; introduzca variables globales o arreglos
. code
    •
```

```
main PROC
; introduzca instrucciones
exit
main ENDP
```

```
;Otros procedimientos o funciones
Ejemplo PROC
;instrucciones del procedimiento
Ret
Ejemplo ENDP
END main
```

Como ejemplo, se explica cómo realizar un programa en Assembler, que consiste en escribir, una secuencia de instrucciones que sumen un bloque de memoria que comienza en la dirección de memoria 0030_H hasta la dirección de memoria 0040_H y deposite el resultado en la dirección de memoria 0040_H.

En la Figura 4.29 está el código nemónico donde se indican los valores del Registro CS (recuadro en rojo), el offset (recuadro en verde) y las instrucciones de cada línea de instrucciones.

Se debe sumar un bloque de memoria que comienza en 0030_H hasta 0040_H y depositar el resultado de 0040_H, por lo que la primera instrucción consiste en cargar el Registro BX con el valor 30_H para hacer referencia luego a su contenido.

La segunda instrucción permite inicializar el Registro AX en cero (haciendo en este la operación XOR sobre sí mismo que permite colocar su valor en 0).

En la tercera instrucción se suma al Registro AX el contenido de la posición de memoria cargado en el Registro BX (por eso la utilización de los corchetes que hace referencia al contenido del Registro BX, esto es, la posición de memoria 30_H). El resultado queda en el acumulador o registro AX.

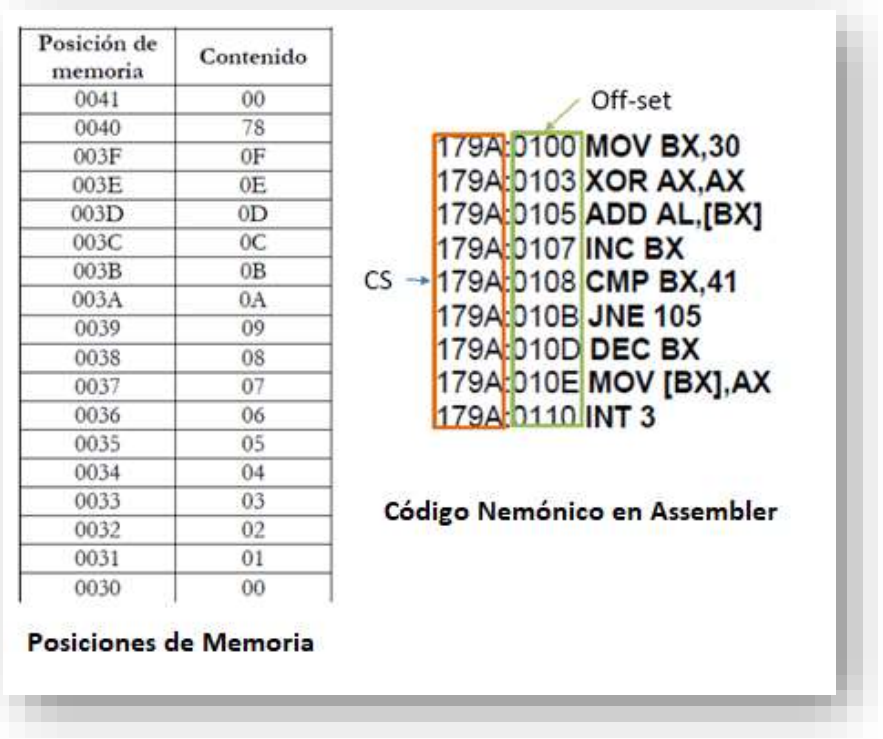


Figura 4.29 Posiciones de Memoria y código Nemónico para el ejemplo de programa

En la cuarta instrucción, se incrementa el valor del Registro BX en una unidad que hace que el valor el Registro ahora sea 31_H, de este modo se comienzan a recorrer las diferentes posiciones de memoria que se deben sumar.

En la quinta instrucción, se compara si el valor del registro BX es igual al valor 41_H que sería una unidad más que el valor de la lista solicitada en la especificación, a los efectos de saber si se llegó al final de la lista a sumar correspondiente.

La siguiente es una instrucción de control de flujo, un salto condicional que realizará la operación de salto al valor 105 de off set mientras el valor del Registro BX no sea igual al valor 41_H. Esto sucederá para completar la cantidad de posiciones solicitadas a sumar. Cuando ambos valores sean iguales se ignora el salto al valor 105 de off set y se continúa con la instrucción siguiente que consiste en decrementar el valor del Registro BX en una unidad para asignarle el valor 40_H.

Esta última operación permite que la instrucción siguiente mueva el contenido del acumulador (Registro AX que contiene la suma de cada uno de los valores comprendidos entre las direcciones 30_H y 40_H debido a la iteración del salto condicional) a la posición de memoria 40_H como se solicitó al inicio.

Obsérvese en la figura 4.30 que el Registro CS tiene un valor de $179A:0100_H$ que es a partir de donde se comienza a programar. El off set de cada valor a medida que vamos programando corresponde al código de máquina asignado a cada operación según su longitud. Obsérvese que de la dirección inicial 179A:100 al cargar la instrucción MOV BX,30, el siguiente valor de offset es 179A:0103 ya que el código máquina asignado a la primera instrucción es BB3000 en hexadecimal que corresponde al byte 179A:0100 correspondiente al código BB, el byte

179A:0101 corresponde al código 30 y el byte 179A:0102 correspondiente al código 00, razón por la cual la segunda instrucción XOR AX,AX comienza en el byte 179A:0103.

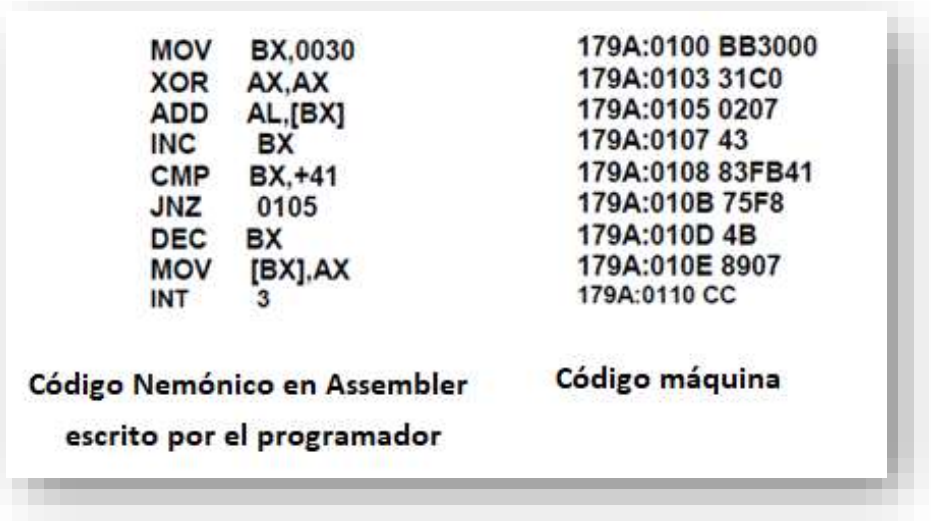


Figura 4.30 Códigos Nemónico y de Máquina del Programa ejemplo

4.3.1 Ejemplo de elementos del Lenguaje Ensamblador

Una instrucción en un lenguaje ensamblador típico consta de cuatro elementos: etiqueta, mnemónico, operando y comentario.

Etiqueta

Si hay una etiqueta, el ensamblador la define como equivalente a la dirección donde se cargará el primer byte del código objeto generado para esa instrucción.

Posteriormente, el programador puede usar la etiqueta como dirección o como datos en el campo de dirección de otra instrucción. El ensamblador reemplaza la etiqueta con el valor asignado al crear un programa objeto. Las etiquetas se utilizan con mayor frecuencia en las instrucciones de bifurcación.

A modo de ejemplo, a continuación, se muestra un fragmento de programa:

```
L2:     SUB EAX, EDX        ; restar el contenido del registro EDX del contenido de EAX y
                            ; almacenar el resultado en EAX
        JG  L2              ; saltar a L2 si el resultado de la resta es positivo
```

El programa continuará regresando a la ubicación L2 hasta que el resultado sea cero o negativo. Por lo tanto, cuando se ejecuta la instrucción jg, si el resultado es positivo, el procesador coloca la dirección equivalente a la etiqueta L2 en el contador del programa.

Las razones para utilizar una etiqueta incluyen las siguientes:

1. Una etiqueta hace que la ubicación de un programa sea más fácil de encontrar y recordar.
2. La etiqueta se puede mover fácilmente para corregir un programa. El ensamblador automáticamente cambia la dirección en todas las instrucciones que utilizan la etiqueta cuando el programa se vuelve a montar.
3. El programador no tiene que calcular direcciones de memoria relativa o absoluta, solo usa etiquetas según sea necesario.

Mnemonico

El mnemónico es el nombre de la operación o función del conjunto en declaración del lenguaje assembler. Como se analiza más adelante, una declaración puede corresponder a una instrucción de máquina, una directiva de ensamblador o una macro. En el caso de una instrucción de máquina, un mnemotécnico es el nombre simbólico asociado con un código de operación particular.

La Tabla 4.30 enumera el mnemónico o nombre de instrucción de algunas de las instrucciones x86.

Etiqueta	Mnemónico	Operando(s)	; comentario
Opcional	Nombre del código de operación o nombre de la directiva o nombre de la macro	Cero o más Opcional	

Operando(S)

Una declaración en lenguaje ensamblador incluye cero o más operandos, cada operando identifica un valor inmediato, un valor de registro o una ubicación de memoria, el lenguaje ensamblador proporciona convenciones para distinguir entre los tres tipos de referencias de operandos, así como convenciones para indicar el modo de direccionamiento.

Para la arquitectura x86, una declaración en lenguaje ensamblador puede hacer referencia a un registro por su nombre. La Figura 4.3.2. Registros internos de los procesadores de la familia Intel x86 de 16 bits (Página 12), ilustra los registros x86 de propósito general, con su nombre simbólico y su codificación de bits. El ensamblador traducirá el nombre simbólico en el identificador binario del registro.

La arquitectura x86 tiene un rico conjunto de modos de direccionamiento, cada uno de los cuales debe expresarse simbólicamente en el lenguaje ensamblador. Aquí citamos algunos de los ejemplos comunes. Para el direccionamiento de registro, el nombre del registro se utiliza en la instrucción.

Por ejemplo,

MOV ECX, EBX copia el contenido del registro EBX en el registro ECX. El direccionamiento inmediato indica que el valor es codificado en la instrucción.

Por ejemplo,

MOV EAX, 100H copia el formato hexadecimal valor 100 en el registro EAX. El valor inmediato se puede expresar como un número binario con el sufijo B o un número decimal sin sufijo.

Así, declaraciones equivalentes al anterior son

MOV EAX, 100000000B y

MOV EAX, 256.

El direccionamiento Directo se refiere a una ubicación de memoria y se expresa como un desplazamiento desde el Registro de segmento DS.

Esto se explica mejor con un ejemplo, supongamos que los datos de 16 bits, el registro de segmento DS contiene el valor 1000H. Entonces ocurre la siguiente secuencia:

MOV AX, 1234H

MOV [3518H], AX

Primero, el registro AX de 16 bits se inicializa a 1234H. Luego, en la segunda línea, el contenido de AX se mueve a la dirección lógica DS:3518H. Esta dirección se forma desplazando el contenido de DS 4 bits a la izquierda y añadiendo 3518H para formar la dirección lógica de 32 bits 13518H.

Comentario

Todos los lenguajes ensambladores permiten la inclusión de comentarios en el programa. Un comentario puede aparecer al final derecho de una sentencia de ensamblado u ocupar una línea de texto completa. En cualquier caso, el comentario comienza con un carácter especial que indica al ensamblador que el resto de la línea es un comentario y debe ser ignorado. Normalmente, los lenguajes ensambladores para la arquitectura x86 utilizan un punto y coma (;) como carácter especial.

4.4. Enlazadores y Cargadores

El software generalmente consiste de varios módulos que son compilados o ensamblados independientemente y como resultado se obtienes programas objetos separados.

Todas las direcciones en los programas objetos están referenciadas a una dirección absoluta “0”

Los módulos tienen referencias a variables o direcciones simbólicas en otros módulos o utilizan funciones provenientes de librerías externas.

Estas referencias son resueltas por los enlazadores o linkers que generan como resultado un módulo objeto único.

El resultado del proceso de enlazado es almacenado en una memoria externa no volátil y finalmente debe ser ubicado en memoria principal, para su ejecución, para lo cual se utiliza otro programa llamado cargador.

Muchas veces un mismo programa realiza ambas funciones denominándose en consecuencia enlazador-cargador.



Figura 4.36 El camino hacia un archivo ejecutable

En la Figura 4.36 pueden observarse los pasos en el proceso de obtención de un archivo ejecutable a partir de un programa fuente en Assembler o .asm.

La extensión .asm se refiere a un archivo de código fuente escrito en lenguaje ensamblador (Assembly Language) que contiene instrucciones de muy bajo nivel, mnemotécnicas, que un ensamblador traduce directamente a lenguaje máquina (archivo objeto) para ser enlazados por un Enlazador y producir así el archivo ejecutable que será cargado en memoria por el programa Cargador y de este modo quedar disponible para su ejecución por el microprocesador.

En la Figura 4.37 pueden observarse los pasos en el proceso de obtención de un archivo ejecutable a partir de un programa escrito en un Lenguaje de Alto Nivel (LAN) para su ejecución en el procesador.

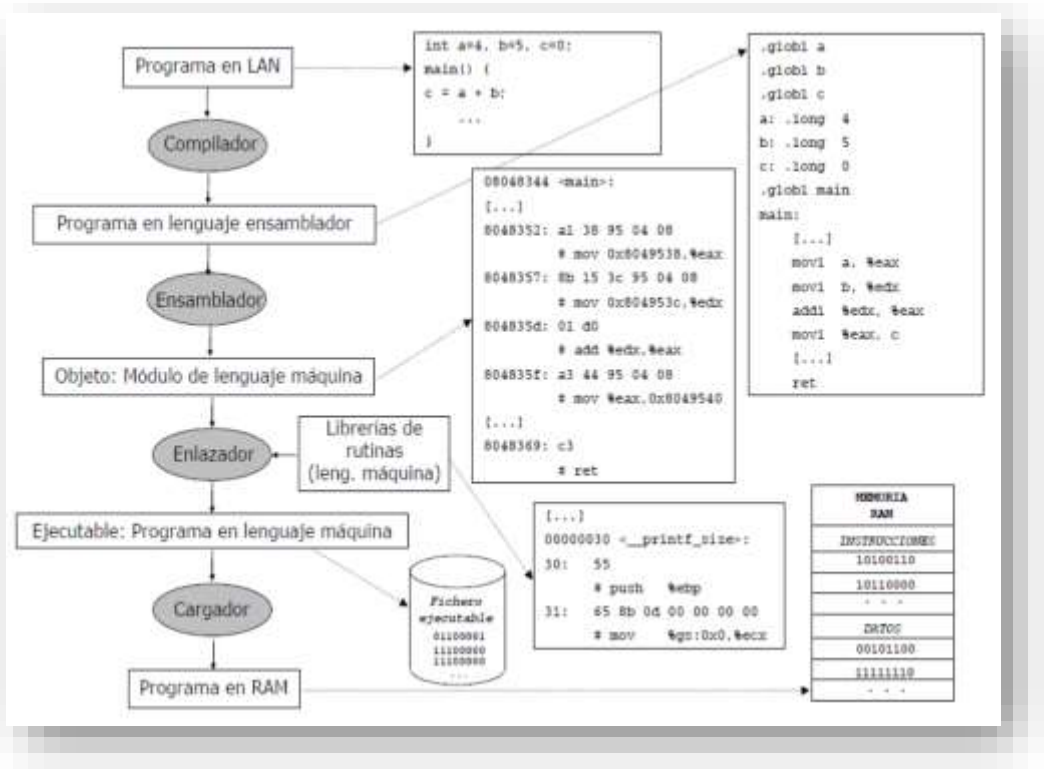


Figura 4.37 Camino desde el Programa en Lenguaje de Alto Nivel (LAN) al programa que se ejecuta en el Microprocesador (Programa en RAM).

AUTOESTUDIO

1. ¿Cuál es la función principal de un ensamblador?

A) Ejecutar programas escritos en lenguaje de alto nivel.
B) Traducir un programa escrito en lenguaje ensamblador a código máquina.
C) Convertir código máquina en lenguaje ensamblador.
D) Administrar la memoria del sistema.

2. ¿Qué característica distingue al lenguaje ensamblador del lenguaje de máquina?

A) Es independiente del procesador.
B) Utiliza únicamente números binarios.
C) Emplea símbolos o nemónicos para representar instrucciones de máquina.
D) Se ejecuta directamente sin traducción.

3. ¿Cuál de los siguientes NO es un tipo de traductor de lenguajes?

A) Ensamblador.
B) Compilador.
C) Intérprete.
D) Depurador.

4. ¿Qué hace un intérprete?

A) Genera un programa objeto antes de ejecutar el programa.
B) Traduce todo el programa antes de ejecutarlo.
C) Ejecuta las instrucciones del programa fuente una por una sin generar código objeto.
D) Convierte programas objeto en ejecutables.

5. ¿Cuál es una función principal de los registros internos del procesador?

A) Almacenar archivos permanentemente.
B) Guardar información temporal durante el procesamiento.
C) Sustituir la memoria RAM.
D) Administrar dispositivos de entrada y salida.

6. ¿Qué bandera (flag) indica que el resultado de una operación es cero?

A) CF.
B) ZF.
C) OF.
D) SF.

7. ¿Qué registro de propósito general es conocido como acumulador principal?

A) BX.
B) CX.
C) AX.
D) DX.

8. ¿Qué registro se utiliza habitualmente como contador?

A) AX.
B) BX.
C) CX.
D) DX.

9. ¿Cuál es la función principal del registro IP?

A) Guardar datos temporales.
B) Indicar la dirección de la siguiente instrucción a ejecutar.
C) Contener la dirección del segmento de datos.
D) Almacenar el resultado de operaciones aritméticas.

10. En el modo de direccionamiento inmediato:

A) El dato está almacenado en memoria.
B) El dato se encuentra dentro de la propia instrucción.
C) El dato se obtiene desde un registro índice.
D) Se utiliza una dirección relativa.

11. ¿Qué símbolo identifica una dirección de memoria en el modo de direccionamiento directo?

A) ( )
B) { }
C) [ ]
D) < >

12. En el modo de direccionamiento por registro, el operando se encuentra en:

A) La memoria principal.
B) El registro de estado.
C) Un registro interno del procesador.
D) La pila.

13. ¿Cuál es la principal ventaja del direccionamiento indirecto por registro?

A) Reducir el tamaño del programa.
B) Facilitar el recorrido de vectores o listas en memoria.
C) Eliminar el uso de registros.
D) Evitar el uso del acumulador.

14. ¿Qué ocurre en una instrucción con direccionamiento implícito?

A) Siempre requiere dos operandos.
B) Los operandos están implícitos en la propia instrucción o no son necesarios.
C) Siempre utiliza memoria externa.
D) Necesita un desplazamiento.

15. ¿Qué estilo de almacenamiento utiliza la arquitectura Intel x86?

A) Big-endian.
B) Little-endian.
C) Middle-endian.
D) Segment-endian.

16. ¿Cuál de las siguientes es una operación lógica?

A) MUL.
B) DIV.
C) XOR.
D) INC.

17. ¿Qué hace una instrucción de salto condicional?

A) Siempre cambia el flujo del programa.
B) Cambia el flujo únicamente cuando se cumple una condición.
C) Incrementa automáticamente el registro AX.
D) Finaliza el programa.

18. ¿Cuál es la finalidad de una instrucción CALL?

A) Reiniciar el procesador.
B) Eliminar la pila.
C) Invocar un procedimiento guardando la dirección de retorno.
D) Inicializar registros.

19. ¿Cuál de los siguientes elementos de una instrucción ensamblador identifica una posición de memoria para saltos?

A) Operando.
B) Etiqueta.
C) Comentario.
D) Nemónico.

20. ¿Cuál es la función del enlazador (Linker)?

A) Traducir lenguaje ensamblador a código máquina.
B) Combinar módulos objeto para generar un único archivo ejecutable.
C) Ejecutar directamente el código fuente.
D) Reservar espacio para la pila.

Respuestas:

Pregunta	Opción correcta		Pregunta	Opción correcta
1	B		11	C
2	C		12	C
3	D		13	B
4	C		14	B
5	B		15	B
6	B		16	C
7	C		17	B
8	C		18	C
9	B		19	B
10	B		20	B

BIBLIOGRAFIA

[1] William Stalling (2022). Organización y Arquitectura de Computadores. Editorial Pearson - 11a edición.

[2] William Stallings, Prentice Computer Organization and Architecture Designing for Performance, Eighth Edition, Prentice Hall, Copyright © 2010, 2006 by Pearson Education, Inc., Upper Saddle River, New Jersey, 07458.

[3] Bartlett, J. Programming from the Ground Up. 2003.

[4] Beck, L. System Software. Reading, MA: Addison-Wesley, 1997.

[5] Carter, P. PC Assembly Language. July 23, 2006.

[6] Clarke, D., and Merusi, D. System Software Programming: The Way Things Work. Upper Saddle River, NJ: Prentice Hall, 1998.

[7] Fog, A. Optimizing Subroutines in Assembly Language: An Optimization Guide for x86 Platforms. Copenhagen University College of Engineering, 2008. http://www.agner.org/optimize/

[8] Knaggs, P. and Welsh, S. ARM: Assembly Language Programming. Bournemouth University School of Design, Engineering & Computing. August 31, 2004.

[9] Levine, J. Linkers and Loaders. San Francisco: Morgan Kaufmann, 2000.

[10] Salomon, D. Assemblers and Loaders. Ellis Horwood Ltd, 1993.

[11] Giró J., Vázquez J., Meloni B. y Constable L. “Lenguajes Formales y Teoría de Autómatas”. Editorial Alfaomega.

[12] Intel Corporation. (1982). *Intel 8086 Family User's Manual*. Santa Clara, CA: Intel Corporation